

Document 2 — The Mathematics, as a Problem Set

SIH26166 · Every derivation your terrain pipeline needs, posed in JEE / ISI-CMI / IMO / STEP style, each followed by why it's in your codebase

Drafted August 31, 2026

Contents

How to read this document	2
Problem 1 [JEE-style] — Sub-pixel keypoint localization by Taylor expansion	2
Problem 2 [ISI/CMI-style] — Homography as a null-space problem	2
Problem 3 [STEP-style, guided] — RANSAC as a probability calculation	3
Problem 4 [JEE-style, 3D vectors] — Stereo triangulation as ray intersection	4
Problem 5 [IMO-flavour existence/inequality] — The Cramér–Rao bound on localization precision	4
Problem 6 [ISI/CMI-style, analysis] — Mutual information as a similarity statistic, and why it's well-defined	5
Problem 7 [STEP-style, guided, Fourier series] — The terrain surface as a truncated 2D Fourier series	6
Problem 8 [JEE-style, coordinate geometry] — Sub-pixel contour placement by linear interpolation (marching squares)	7
Problem 9 [ISI/CMI-style, linear algebra] — Collinearity and the pixel-to-ground transform	7
Problem 10 [STEP-style] — Why the Gaussian pyramid's blur-and-downsample step is the right one, not an arbitrary choice	8
Quick reference — problem-to-codebase map	8

How to read this document

Each entry is written as an exam problem in the house style of the named exam, followed by a full solution, followed by a short “**Where this lives in the project**” note. Nothing here is decorative — every problem is a piece of math your registration/terrain pipeline actually runs. Style key: **[JEE]** — single-concept, calculus/vectors/coordinate geometry; **[ISI/CMI]** — proof-based, linear algebra/probability/analysis; **[STEP]** — multi-step guided derivation; **[IMO]** — a genuine olympiad-flavour argument (inequality/existence), used where the underlying math really is that flavour (e.g. the Cramér–Rao bound, information inequalities).

Problem 1 [JEE-style] — Sub-pixel keypoint localization by Taylor expansion

Problem. A function $D(x, y, \sigma)$ (the Difference-of-Gaussians response) has a discrete maximum at grid point $\mathbf{x}_0 = (x_0, y_0, \sigma_0)$. Using a second-order Taylor expansion of D about $\mathbf{x}_0$, find the offset $\hat{\mathbf{x}}$ that locates the true (continuous) extremum, and express it in terms of the gradient and Hessian of D at $\mathbf{x}_0$.

Solution. Write $\mathbf{x} = \mathbf{x}_0 + \Delta\mathbf{x}$. Then

$$D(\mathbf{x}_0 + \Delta\mathbf{x}) \approx D(\mathbf{x}_0) + \frac{\partial D}{\partial \mathbf{x}}^\top \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^\top \frac{\partial^2 D}{\partial \mathbf{x}^2} \Delta\mathbf{x}.$$

Differentiate with respect to $\Delta\mathbf{x}$ and set to zero (the usual first-order condition for a stationary point, exactly as in single-variable calculus where $f'(x) = 0$ locates an extremum, just vector-valued here):

$$\frac{\partial D}{\partial \mathbf{x}} + \frac{\partial^2 D}{\partial \mathbf{x}^2} \Delta\mathbf{x} = 0 \implies \hat{\mathbf{x}} = - \left(\frac{\partial^2 D}{\partial \mathbf{x}^2} \right)^{-1} \frac{\partial D}{\partial \mathbf{x}}.$$

The Hessian here is 3×3 (in x, y, σ), so this is one matrix inversion per candidate keypoint — a closed, exact computation given the local pixel values used to estimate the derivatives by finite differences.

Where this lives in the project. `src/matching/sift_refine.py` — every SIFT keypoint you extract from an OHRC/TMC-2 tile gets refined this way before being handed to the descriptor stage; it’s the reason your reported match locations can claim sub-pixel accuracy at all instead of being locked to the integer pixel grid.

Problem 2 [ISI/CMI-style] — Homography as a null-space problem

Problem. Let $(\mathbf{x}_i, \mathbf{x}'_i)$, $i = 1, \dots, n$ ($n \geq 4$), be pairs of corresponding points (in homogeneous coordinates) between two images of a planar scene, related by an unknown 3×3 homography H : $\mathbf{x}'_i \times H\mathbf{x}_i = \mathbf{0}$. Show that estimating H reduces to finding the null space of a $2n \times 9$ matrix, and explain why, in the presence of noise, the natural estimator is the right singular vector of that matrix associated with its smallest singular value.

Solution. Each correspondence gives two independent linear equations in the 9 unknown entries of H (flattened as $\mathbf{h} \in \mathbb{R}^9$) — the cross product $\mathbf{x}'_i \times H\mathbf{x}_i = \mathbf{0}$ has 3 components but only 2 are independent since $\mathbf{x}'_i \times H\mathbf{x}_i$ is always orthogonal to $\mathbf{x}'_i$. Stacking all n correspondences gives $A\mathbf{h} = \mathbf{0}$ with $A \in \mathbb{R}^{2n \times 9}$. Exactly (noise-free): $\mathbf{h} \in \ker(A)$, a 1-dimensional subspace when $n \geq 4$ generic points are given (this is why 4 points are the minimum: $2n \geq 8$ constraints for 8 degrees of freedom, since H is defined only up to scale).

With noise, A generically has full rank ($\ker(A) = \{0\}$), so there is no exact solution; instead solve

$$\hat{\mathbf{h}} = \arg \min_{\|\mathbf{h}\|=1} \|A\mathbf{h}\|^2.$$

Write the SVD $A = U\Sigma V^\top$. Then $\|A\mathbf{h}\|^2 = \|U\Sigma V^\top \mathbf{h}\|^2 = \|\Sigma V^\top \mathbf{h}\|^2$ (since U is orthogonal and preserves norm). Substituting $\mathbf{y} = V^\top \mathbf{h}$ (also unit norm, since V is orthogonal), this is $\sum_j \sigma_j^2 y_j^2$, minimized subject to $\sum y_j^2 = 1$ by putting all the weight on the coordinate with the *smallest* σ_j — i.e. $\mathbf{y} = \mathbf{e}_{\text{last}}$, so $\hat{\mathbf{h}}$ is the last column of V . This is a direct, textbook constrained-optimization argument (Rayleigh-quotient minimization), not a heuristic.

Where this lives in the project. `src/verify/dlt_homography.py`, wrapped in RANSAC (`src/verify/ransac.py`) for outlier rejection. This is Stage 6 of your pipeline and the single most-called linear algebra routine in the codebase.

Problem 3 [STEP-style, guided] — RANSAC as a probability calculation

(i) Suppose a correspondence set has inlier fraction w (probability any single correspondence is a true match). A minimal sample of s points is drawn at random. Show the probability that *all* s points in one sample are inliers is w^s .

(ii) Hence show that the probability that a single random sample is **not** all-inlier is $1 - w^s$, and that after k independent samples, the probability that *none* of them was all-inlier is $(1 - w^s)^k$.

(iii) Given a desired confidence p (e.g. $p = 0.99$) that at least one sample was all-inlier, show

$$k \geq \frac{\log(1 - p)}{\log(1 - w^s)}.$$

(iv) For $s = 4$ (homography) and $w = 0.3$ (a noisy shadow-affected correspondence set), compute the minimum k for $p = 0.99$.

Solution. (i) Draws are independent, each inlier with probability w , so $P(\text{all } s \text{ inliers}) = w^s$ by the multiplication rule for independent events. (ii) Complement rule gives $1 - w^s$ per sample; independence across the k samples gives $(1 - w^s)^k$ for “none succeeded.” (iii) We require $(1 - w^s)^k \leq 1 - p$. Taking logarithms (both sides positive, log monotonic, and $\log(1 - w^s) < 0$ so the inequality flips):

$$k \log(1 - w^s) \leq \log(1 - p) \implies k \geq \frac{\log(1 - p)}{\log(1 - w^s)}.$$

(iv) $w^4 = 0.3^4 = 0.0081$. $\log(1 - 0.99) = \log(0.01) \approx -4.605$. $\log(1 - 0.0081) = \log(0.9919) \approx -0.00813$. So $k \geq 4.605/0.00813 \approx 566.5$, i.e. $k = 567$ **minimal samples** — this is exactly why

RANSAC iteration counts explode as the shadow-corrupted inlier fraction drops, and exactly the number your logging should report per scene pair so a judge sees you understand *why* your iteration budget is what it is, not just that you set `max_iters=1000` by convention.

Where this lives in the project. `src/verify/ransac.py::estimate_iterations(w, s, p)` — call this function *before* running RANSAC on a real degraded correspondence set and log the estimated w from your matcher’s own inlier history; this turns your RANSAC config from a magic number into a stated, derived one.

Problem 4 [JEE-style, 3D vectors] — Stereo triangulation as ray intersection

Problem. Two cameras with known optical centres $C_1, C_2 \in \mathbb{R}^3$ observe the same ground point P . The corresponding image rays are $\mathbf{r}_1 = C_1 + t_1 \mathbf{d}_1$ and $\mathbf{r}_2 = C_2 + t_2 \mathbf{d}_2$ for unit direction vectors $\mathbf{d}_1, \mathbf{d}_2$. In the presence of noise the two rays do not exactly meet; find the point minimizing the sum of squared perpendicular distances to both rays (the standard “midpoint of common perpendicular” solution), and set up the linear system that gives t_1, t_2 .

Solution. The vector connecting the closest points on the two lines is perpendicular to both directions. Writing $\mathbf{w} = C_1 - C_2$, and requiring

$$(\mathbf{r}_1 - \mathbf{r}_2) \cdot \mathbf{d}_1 = 0, \quad (\mathbf{r}_1 - \mathbf{r}_2) \cdot \mathbf{d}_2 = 0,$$

substituting $\mathbf{r}_1 - \mathbf{r}_2 = \mathbf{w} + t_1 \mathbf{d}_1 - t_2 \mathbf{d}_2$ gives the 2×2 linear system

$$\begin{pmatrix} \mathbf{d}_1 \cdot \mathbf{d}_1 & -\mathbf{d}_1 \cdot \mathbf{d}_2 \\ \mathbf{d}_1 \cdot \mathbf{d}_2 & -\mathbf{d}_2 \cdot \mathbf{d}_2 \end{pmatrix} \begin{pmatrix} t_1 \\ t_2 \end{pmatrix} = \begin{pmatrix} -\mathbf{w} \cdot \mathbf{d}_1 \\ -\mathbf{w} \cdot \mathbf{d}_2 \end{pmatrix},$$

solved directly (Cramer’s rule, familiar from JEE 3D-geometry line problems, extended to skew lines in space). The estimated ground point is the midpoint of the segment joining $\mathbf{r}_1(t_1)$ and $\mathbf{r}_2(t_2)$; this midpoint’s height above a reference sphere/ellipsoid is your recovered terrain elevation $z(x, y)$ at that ground location.

Where this lives in the project. `src/geometry/triangulate.py`, applied across TMC-2’s fore/nadir/aft stereo triplet correspondences (once matched via Stage 5/6) to build your own DEM patch where an external one isn’t available for the exact scene you chose.

Problem 5 [IMO-flavour existence/inequality] — The Cramér–Rao bound on localization precision

Problem. Let $I(x)$ be the noiseless image intensity along a 1D scanline near a feature, and suppose the *observed* intensity is $I(x) + \eta$, with $\eta \sim \mathcal{N}(0, \sigma^2)$ i.i.d. per pixel. An estimator $\hat{x}$ locates the feature from N noisy pixel samples. Prove that **no unbiased estimator** can have variance smaller than

$$\text{Var}(\hat{x}) \geq \frac{\sigma^2}{\sum_{k=1}^N \left(\frac{\partial I}{\partial x}(x_k) \right)^2}.$$

Solution sketch (Fisher information argument). The log-likelihood of observing samples $\{y_k\}$ given true location x is

$$\ell(x) = -\frac{1}{2\sigma^2} \sum_k (y_k - I(x_k - x))^2 + \text{const.}$$

Differentiating twice with respect to x and taking the expectation (using $\mathbb{E}[y_k] = I(x_k - x)$ so the second-derivative cross-terms vanish in expectation) gives the Fisher information

$$\mathcal{I}(x) = -\mathbb{E}\left[\frac{\partial^2 \ell}{\partial x^2}\right] = \frac{1}{\sigma^2} \sum_k \left(\frac{\partial I}{\partial x}\right)^2.$$

The Cramér–Rao inequality (a genuine information-theoretic *lower bound*, proved in general via the Cauchy–Schwarz inequality applied to the score function and the estimator’s derivative — the IMO-style move: bound a variance below by squaring a correlation and invoking Cauchy–Schwarz) states $\text{Var}(\hat{x}) \geq 1/\mathcal{I}(x)$, giving exactly the claimed bound.

Interpretation. A large local gradient $\partial I/\partial x$ (a sharp crater rim) drives the bound down — precise localization is *possible*. A flat, textureless regolith patch has $\partial I/\partial x \approx 0$ everywhere, and the bound blows up — **no algorithm, however clever, can localize a feature there better than this bound allows**. This is the honest, information-theoretic ceiling your own workplan names as an open question (Log 04/07) for the specific OHRC/TMC-2/IIRS instrument pairing — deriving the *general form* here is legitimate; deriving the *exact numeric bound for your specific sensor noise* is real, uncomputed work you’d do empirically per scene, not something to state as an already-known number.

Where this lives in the project. Cite this bound in your evaluation write-up (Stage 8) whenever your measured RMSE approaches the image’s local gradient-content limit — it’s the correct, honest explanation for “why can’t you get RMSE lower,” and it’s a materially stronger answer than “we ran out of time.”

Problem 6 [ISI/CMI-style, analysis] — Mutual information as a similarity statistic, and why it’s well-defined

Problem. For two random variables A, B with joint distribution $p(a, b)$ and marginals $p(a), p(b)$, mutual information is defined as

$$I(A; B) = \sum_{a,b} p(a, b) \log \frac{p(a, b)}{p(a)p(b)}.$$

Prove $I(A; B) \geq 0$, with equality iff A, B are independent.

Solution. Write $I(A; B) = \sum_{a,b} p(a, b) \log \frac{p(a,b)}{p(a)p(b)} = -\sum_{a,b} p(a, b) \log \frac{p(a)p(b)}{p(a,b)}$. By Jensen’s inequality applied to the (strictly) concave function $\log$:

$$-\sum_{a,b} p(a, b) \log \frac{p(a)p(b)}{p(a,b)} \geq -\log \left(\sum_{a,b} p(a, b) \cdot \frac{p(a)p(b)}{p(a,b)} \right) = -\log \left(\sum_{a,b} p(a)p(b) \right) = -\log(1) = 0,$$

using $\sum_{a,b} p(a)p(b) = 1$. Equality in Jensen's inequality holds iff the argument $\frac{p(a)p(b)}{p(a,b)}$ is constant over all (a,b) with $p(a,b) > 0$, which (after normalizing) forces $p(a,b) = p(a)p(b)$ everywhere — i.e. independence. This is a completely standard, fully rigorous proof — mutual information as a *number* is textbook. (What is genuinely open, per your own Log 04 Open 3, is whether it's the *right* statistic for panchromatic-vs-hyperspectral matching specifically — that's an empirical question about your data, not a gap in the proof above.)

Where this lives in the project. `src/matching/mutual_information.py`, used for the IIRS cross-modal stretch goal (Stage 5, cross-modal branch): build the joint histogram of a candidate patch pair's intensities (or a chosen spectral-index projection for IIRS), normalize, evaluate the sum directly.

Problem 7 [STEP-style, guided, Fourier series] — The terrain surface as a truncated 2D Fourier series

(i) State the 2D Fourier series representation of a doubly-periodic function $z(x,y)$ on a rectangle $[0, L_x] \times [0, L_y]$, and give the formula for the coefficients c_{mn} in terms of z .

(ii) Explain why truncating the sum to $|m| \leq M, |n| \leq N$ acts as a low-pass filter, and why this is exact given the sampled DEM data (no fitting uncertainty).

(iii) Show that removing a narrow band of frequencies corresponding to a known periodic scan-line artifact, then inverting, removes that artifact without needing to know its amplitude in advance.

Solution. (i)

$$z(x,y) = \sum_{m=-M}^M \sum_{n=-N}^N c_{mn} e^{i2\pi(mx/L_x + ny/L_y)}, \quad c_{mn} = \frac{1}{L_x L_y} \int_0^{L_x} \int_0^{L_y} z(x,y) e^{-i2\pi(mx/L_x + ny/L_y)} dx dy,$$

the direct 2D analogue of the standard 1D Fourier series orthogonality argument: multiplying both sides by $e^{-i2\pi(m'x/L_x + n'y/L_y)}$ and integrating over the rectangle, every term except $m = m', n = n'$ vanishes by orthogonality of complex exponentials on a period, isolating $c_{m'n'}$.

(ii) For sampled data, the coefficients are computed exactly by the discrete Fourier transform (`numpy.fft.fft2`) — there is no fitting error, only the deliberate choice of how many terms (M, N) to keep. Discarding high- $|m|, |n|$ terms is precisely a low-pass filter: those terms carry the rapidly-varying (high-spatial-frequency) content, which for a DEM patch is dominated by pixel-level noise rather than the smooth macroscopic terrain shape you want for a clean render.

(iii) A periodic scan-line artifact of known spatial period T contributes essentially all its energy to one specific (m,n) bin (or a narrow band around it) in the frequency domain, by the same orthogonality argument as (i) — a pure sinusoid of period T is (to good approximation) a single Fourier coefficient. Zeroing exactly that bin (or a small neighborhood, to tolerate slight period drift) and inverse-transforming removes the artifact's contribution to $z(x,y)$ without ever needing to estimate its amplitude directly in the spatial domain — the frequency-domain representation isolates it for you.

Where this lives in the project. `src/spectral/fourier_surface.py` (Stage 7/9 of your PDF) — this is the exact data structure fed to the React Three Fiber terrain mesh: a coefficient grid,

evaluated at whatever resolution the renderer needs, re-tessellatable without re-reading the source imagery.

Problem 8 [JEE-style, coordinate geometry] — Sub-pixel contour placement by linear interpolation (marching squares)

Problem. A unit grid cell has corner heights z_1, z_2 at its two ends of an edge, with $z_1 < z_k < z_2$ for a chosen contour level z_k . Find the point P along the edge (parametrized from P_1 to P_2) where the surface crosses height z_k , assuming z varies linearly along the edge.

Solution. Linear interpolation: $z(t) = z_1 + t(z_2 - z_1)$ for $t \in [0, 1]$. Setting $z(t) = z_k$:

$$t = \frac{z_k - z_1}{z_2 - z_1}, \quad P = P_1 + t(P_2 - P_1).$$

This is the same “section formula” idea from JEE coordinate geometry (finding the point dividing a segment in a given ratio), just with the ratio determined by height rather than given directly. Applying this rule to every grid-cell edge that changes sign relative to z_k , and connecting the resulting points according to which of the 16 possible corner sign-patterns the cell falls into, is exactly the marching-squares algorithm.

Where this lives in the project. `skimage.measure.find_contours(height_grid, level=z_k)` in `src/viz/contours.py` — implements exactly the interpolation above; you don’t hand-roll it, but you should be able to derive it live if a judge asks why your contour lines aren’t jagged/pixel-locked.

Problem 9 [ISI/CMI-style, linear algebra] — Collinearity and the pixel-to-ground transform

Problem. A camera at position $\mathbf{C}$ with orientation given by a rotation matrix R (camera-to-world) observes ground point $\mathbf{P}$. In the camera’s own frame, the image-plane coordinates (u, v) of $\mathbf{P}$ ’s projection satisfy $\lambda(u, v, -f)^\top = R^\top(\mathbf{P} - \mathbf{C})$ for some scalar $\lambda > 0$ and focal length f . Show how, given a known ground elevation surface $z = h(x, y)$ (e.g. a DEM), this equation together with the surface constraint determines $\mathbf{P}$ uniquely (generically), and identify exactly which quantities must come from metadata versus which come from the DEM.

Solution. The equation says the ray from $\mathbf{C}$ through the image point, direction $\mathbf{d} = R(u, v, -f)^\top$ (unnormalized), must pass through $\mathbf{P}$: $\mathbf{P} = \mathbf{C} + s\mathbf{d}$ for some $s > 0$. Substituting the third coordinate of $\mathbf{P}$ into the surface constraint $P_z = h(P_x, P_y)$ gives one scalar equation in the one unknown s (generically solvable, e.g. by 1D root-finding since h is a general surface, or in closed form if h is locally planar). Once s is found, $\mathbf{P} = \mathbf{C} + s\mathbf{d}$ is fully determined — a direct ray-surface intersection, the photogrammetric “collinearity” equation. R and $\mathbf{C}$ (the camera’s exterior orientation) come entirely from the orbit/attitude ephemeris in the scene’s metadata; f and the $(u, v) \rightarrow \text{ray}$ mapping come from the sensor’s interior calibration (also in the metadata/PDS4 label); $h(x, y)$ comes from your DEM. **No quantity here is estimated from the image content itself** — this is why it’s classified as closed-form/deterministic geometry rather than an open research question.

Where this lives in the project. `src/geometry/pixel_to_ground.py` and its inverse `ground_to_pixel.py` — used both to reproject a reference image into a source image’s frame for direct comparison, and to convert your final matched keypoints into selenographic lat/lon for reporting.

Problem 10 [STEP-style] — Why the Gaussian pyramid’s blur-and-downsample step is the right one, not an arbitrary choice

Problem. Let G_σ denote convolution with a Gaussian of standard deviation σ . The pyramid construction is $I_{l+1}(x, y) = (G_\sigma * I_l)(2x, 2y)$. Explain, using the sampling theorem, why the Gaussian blur must precede the $2\times$ downsampling, and what goes wrong if you downsample first.

Solution. Downsampling by a factor of 2 halves the maximum representable spatial frequency (the new Nyquist frequency is half the old one). If I_l contains spatial frequencies above the new Nyquist limit and you downsample without first removing them, those frequencies **alias** — they reappear in the downsampled image as spurious *lower*-frequency content indistinguishable from genuine coarse-scale structure (the same phenomenon as a wagon wheel appearing to spin backwards on camera due to frame-rate undersampling). Convoluting with G_σ first is a low-pass filter that suppresses energy above the new Nyquist frequency *before* the irreversible information loss of downsampling occurs, so the coarser pyramid level is a faithful smoothed representation rather than a corrupted one. This is precisely why Problem 7’s Log-03 pyramid step is “blur-then-downsample,” never the reverse, and why matching against an incorrectly-ordered pyramid would silently produce wrong coarse-level correspondences with no error thrown — exactly the class of silent bug your own AI-boundary log warns about.

Where this lives in the project. `src/matching/pyramid.py` (Stage 5 coarse-to-fine matching across the ~17–20:1 OHRC-to-TMC-2 scale ratio) — get the operation order right here or the coarse-to-fine search degrades silently.

Quick reference — problem-to-codebase map

Problem	Concept	Codebase location
1	Taylor expansion, sub-pixel extrema	<code>matching/sift_refine.py</code>
2	SVD null-space, homography	<code>verify/dlt_homography.py</code>
3	Probability, RANSAC iteration count	<code>verify/ransac.py</code>
4	3D vectors, ray intersection	<code>geometry/triangulate.py</code>
5	Fisher information, Cramér–Rao bound	evaluation write-up, Stage 8
6	Jensen’s inequality, mutual information	<code>matching/mutual_information.py</code>
7	2D Fourier series	<code>spectral/fourier_surface.py</code>
8	Linear interpolation, contours	<code>viz/contours.py</code>

Problem	Concept	Codebase location
9	Linear algebra, collinearity	<code>geometry/pixel_to_ground.py</code>
10	Sampling theorem, aliasing	<code>matching/pyramid.py</code>